

Model Architecture Research Summary

Objective

Build a bilingual (Marathi + English) university assistant by training a small language model from scratch and integrating it with Retrieval-Augmented Generation (RAG).

Architectures Explored

1. nanoGPT (GPT-2 Style)

- Decoder-only Transformer architecture.
- Simple, well-documented, and beginner-friendly.
- Suitable for training a 20–25M parameter model on Kaggle.
- Acts as the base architecture for our project.

2. Core Language Model - Custom GPT Transformer

- Architecture

Base: nanoGPT

with improvements: (Llama)

- RMSNorm
- RoPE positional embeddings
- SwiGLU activation

Final Proposed Pipeline

1. Collect university documents.
2. Generate bilingual synthetic Q&A data.
3. Train a custom BPE tokenizer.
4. Build a GPT-style Transformer using nanoGPT.
5. Pretrain the model from scratch using next-token prediction.
6. Perform Supervised Fine-Tuning (SFT) on instruction/Q&A data.
7. Optionally replace the Feed-Forward layer with a simple MoE implementation.
8. Build a RAG pipeline using:
 - a pretrained multilingual sentence embedding model,
 - ChromaDB as the vector database.
9. Connect RAG with the trained model.
10. Develop a chat interface and evaluate the system.

Fine-Tuning Findings

- Since we are training the model from scratch.
- The correct workflow is:
 - Pretraining → Supervised Fine-Tuning (SFT) → RAG.

Recommended Tech Stack

- **Base Model:** nanoGPT (GPT-2 style)
- **Tokenizer:** Hugging Face BPE Tokenizer
- **Training:** PyTorch
- **Fine-Tuning:** Supervised Fine-Tuning (SFT)
- **Knowledge Retrieval:** ChromaDB + pretrained multilingual sentence embedding model
- **UI:** Gradio or Streamlit

Conclusion

After comparing different architectures, nanoGPT is the most practical choice because it is easy to understand, trainable on Kaggle, and provides a strong foundation for learning. Once a working dense model is available, modern improvements such as RoPE, RMSNorm, SwiGLU, and a simplified Mixture-of-Experts (MoE) layer can be added incrementally. This approach balances feasibility with the opportunity to demonstrate advanced LLM concepts in the final project.